EUUI--输入导航处理（单人游戏和多人同屏游戏）

UI的输入处理思路：

Unity UI 事件需要处理：

EventSystem（事件系统）和 **InputModule**（输入模块）

常规的初始化

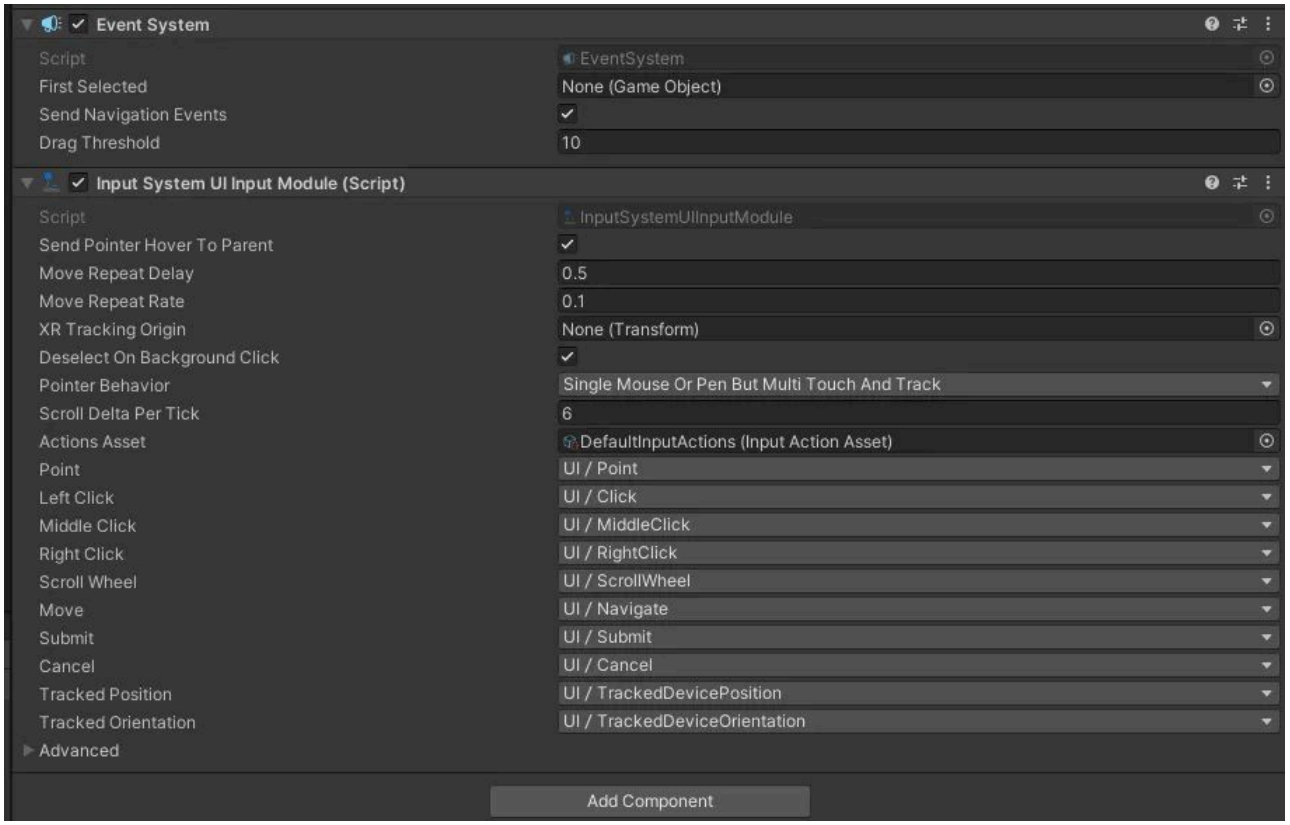
如果是常规的初始化：

C#

```
/// 提供个性化实现，在确保 EventSystem 存在前调用
static partial void OnBeforeEnsureEventSystem();
static partial void OnAfterEnsureEventSystem();
private static void EnsureEventSystem()
{
    OnBeforeEnsureEventSystem();

    if (UnityEngine.EventSystems.EventSystem.current == null)
    {
        Debug.Log("[EUUIKit] 未找到 EventSystem，创建新实例");
        var es = new GameObject("EventSystem");
        es.AddComponent<UnityEngine.EventSystems.EventSystem>();

        bool hasInputSystem = false;
        try
        {
            var inputSystemType = System.Type.GetType(
                "UnityEngine.InputSystem.UI.InputSystemUIInputModule, Unity.InputSystem");
            if (inputSystemType != null)
            {
                es.AddComponent(inputSystemType);
                hasInputSystem = true;
            }
        }
        catch (System.Exception) { }

        if (!hasInputSystem)
            es.AddComponent<UnityEngine.EventSystems.StandaloneInputModule>();

        UnityEngine.Object.DontDestroyOnLoad(es);
```

```
        _globalEventSystemGO = es;
    }
    else
    {
        // 场景中已有 EventSystem，保存引用（此时多人 ES 尚未创建，current 一定是全局单人 ES）
        _globalEventSystemGO = UnityEngine.EventSystems.EventSystem.current.gameObject;
    }

    OnAfterEnsureEventSystem();
}
```

只需要使用默认的EventSystem 和 默认的InputModule （InputSystemInputModule，输入模块默认接受所有的接入设备不做区分）直接使用DefaultInputActionsAssets。



框架也提供使用自己的EventSystem和InputModule，只需要在处理的地方实现下列两个函数的调用（初始化InputModule，Actions Assets和Pont→Tracked Orientation的填入即可）

C#

```
static partial void OnBeforeEnsureEventSystem();
static partial void OnAfterEnsureEventSystem();
```

默认的导航设置好导航图（在UI制作的时候，可以给按钮选择AutoNavigate），以及设置好初始的焦点（Event System中的First Selected）

C#

```
//设置初始焦点 EUUIKit调用
public static void SetDefaultSelection(Selectable selectable)
{
    if (_isMultiplayer) return;
    if (selectable == null || EventSystem.current == null) return;
    EventSystem.current.SetSelectedGameObject(selectable.gameObject);
}

// Panle中只需要重写GetDefaultSelectable  打开时就会自动调用SetDefaultSelection
public class YourPanel : EUUIPanelBase<YourPanel>
{
    [SerializeField] private Button firstFocusButton; // 或直接在代码里 Find

    protected override Selectable GetDefaultSelectable()
    {
        return firstFocusButton; // 返回要默认选中的控件
    }
}
```

除了使用默认的导航图，也可以声明一组导航控件调用PanleBase中的SetupNavigationChain

C#

```
///
/// 将一组 Selectable 连接为循环/非循环导航链
/// vertical=true 为上下链，false 为左右链
///
protected void SetupNavigationChain(bool loop, bool vertical, params Selectable[] selectables)
{
    if (selectables == null || selectables.Length == 0) return;
    for (int i = 0; i < selectables.Length; i++)
    {
        if (selectables[i] == null) continue;
        var nav = selectables[i].navigation;
        nav.mode = Navigation.Mode.Explicit;

        int prev = loop ? (i - 1 + selectables.Length) % selectables.Length : i - 1;
```

```
        int next = loop ? (i + 1) % selectables.Length : i + 1;

        if (vertical)
        {
            nav.selectOnUp = prev >= 0 && prev < selectables.Length ? selectables[prev] : null;
            nav.selectOnDown = next >= 0 && next < selectables.Length ? selectables[next] : null;
        }
        else
        {
            nav.selectOnLeft = prev >= 0 && prev < selectables.Length ? selectables[prev] : null;
            nav.selectOnRight = next >= 0 && next < selectables.Length ? selectables[next] : null;
        }
        selectables[i].navigation = nav;
    }
}
```

要启用导航最重要的是设置焦点

C#

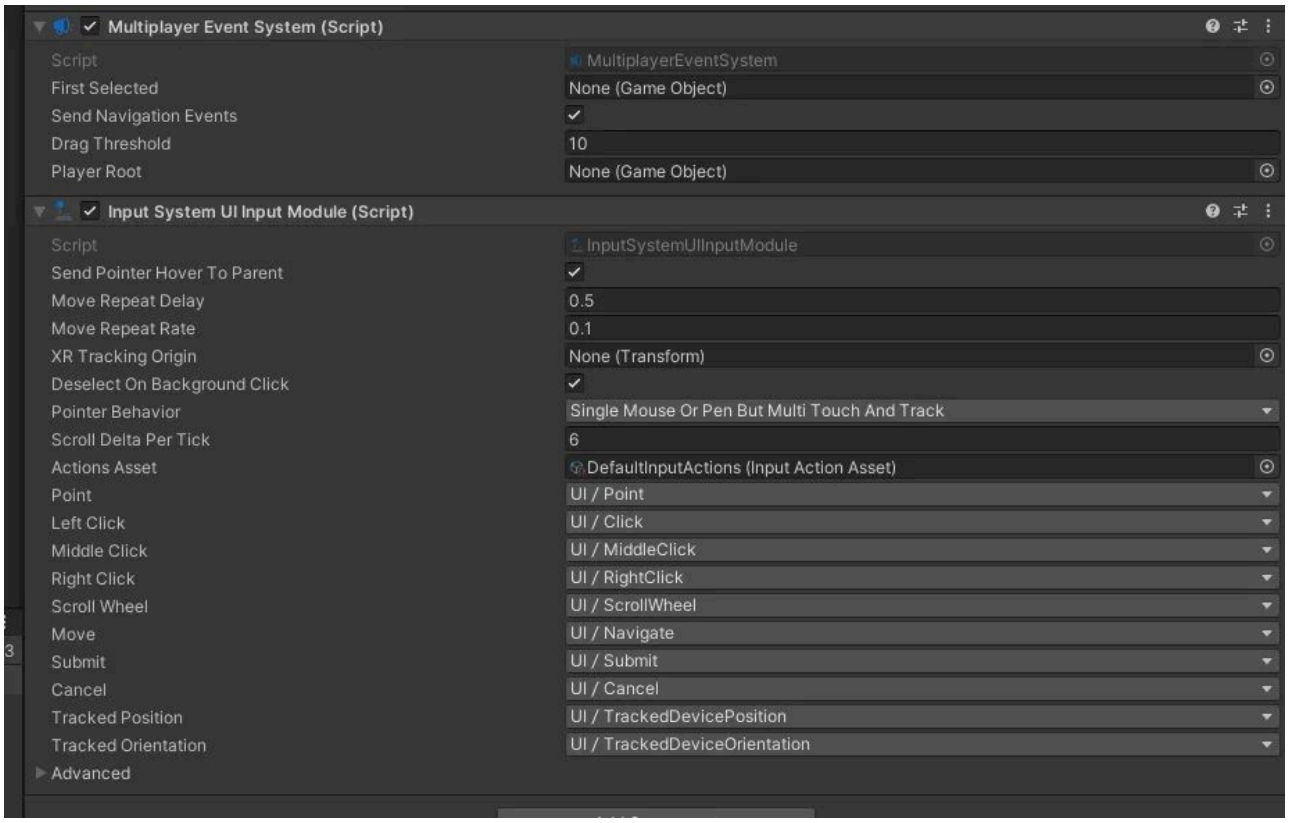
```
        /// <summary>
        /// 面板 Show 时默认聚焦的元素，子类按需重写
        /// 返回 null 则不自动设置焦点
        /// </summary>
        protected virtual Selectable GetDefaultSelectable() => null;
```

面板类直接返回第一个控件即可

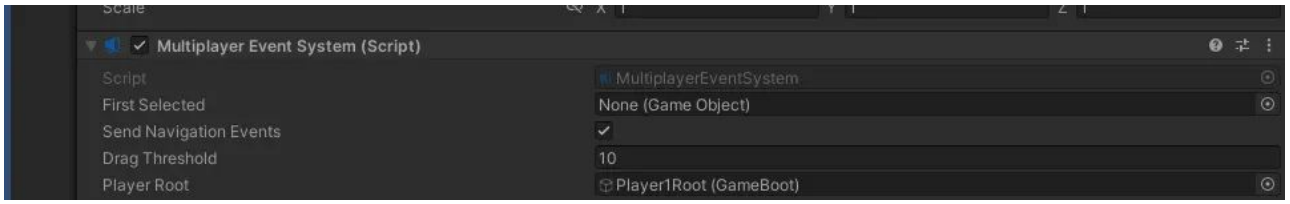
多人的初始化：

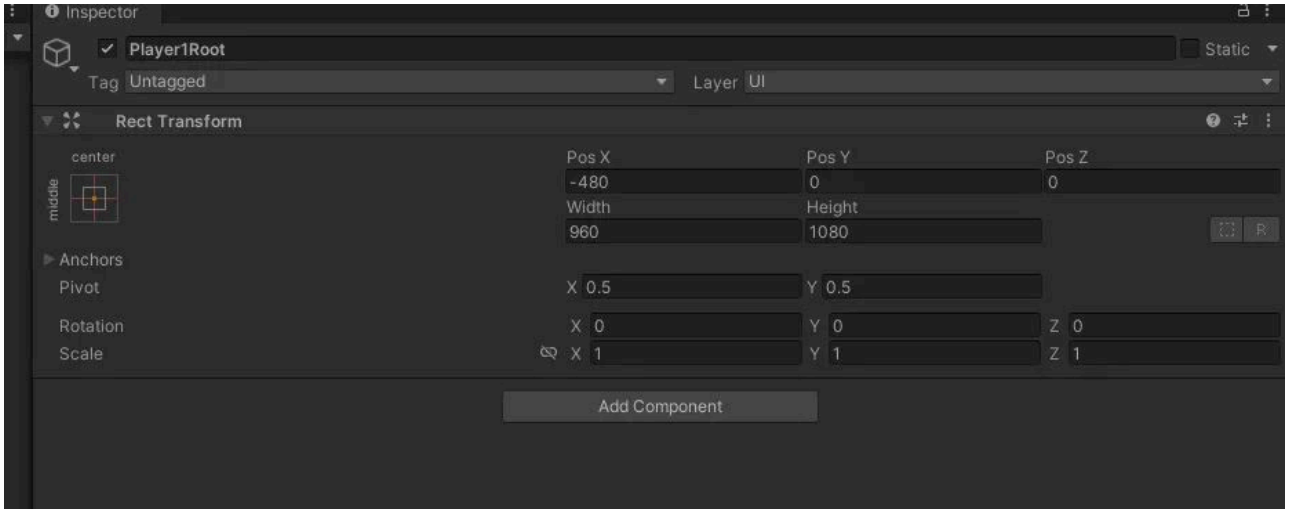
多人同屏游戏除了要初始化常规的单人组件（常用于大厅界面），还必须额外初始化适用于多人玩家的系统。核心目标是让每个玩家的输入能被独立处理和区分。

初始化MultiplayerEventSystem和对应的响应区域



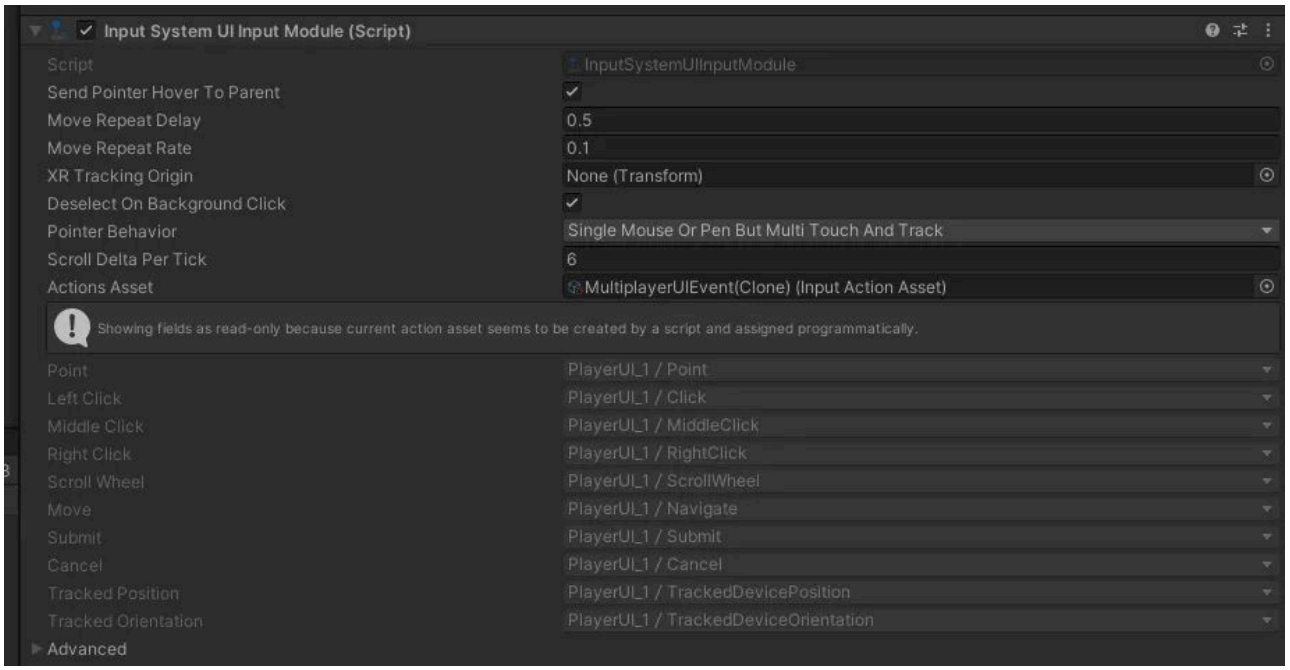
MultiplayerEventSystem 相较于EventSystem只是多了PlayerRoot 响应区域





只有在这个节点下的UI才会收到该MultiplayerEventSystem输入的影响

下一步是处理MultiplayerEventSystem的输入方式，即处理InputAssets和



获取InputAssets（若是在EUInputControl中统一管理则修改加载的接口即可）

C#

```
        private static InputActionAsset LoadMultiplayerUIInputAsset()
        {
            var asset = Resources.Load<InputActionAsset>(MultiplayerUIEventAssetName);
            if (asset == null)
                asset = Resources.Load<InputActionAsset>("EUUI/" + MultiplayerUIEventAssetName);
            return asset;
        }
```

处理EventSystem InputActionAssets来源问题：1.从EUInputController中获取 2.从Resources中直接加载

C#

```
        private static void CreatePlayerEventSystem(int index, InputActionAsset sourceAsset)
        {
            string esName = "EventSystem_Player" + (index + 1) + "Root";
            var esGo = new GameObject(esName);
            UnityEngine.Object.DontDestroyOnLoad(esGo);

            // MultiplayerEventSystem.playerRoot → 将该 EventSystem 的鼠标点击与导航限定在玩家区域内
```

```
            var mpEventSystem = esGo.AddComponent<MultiplayerEventSystem>();
            mpEventSystem.playerRoot = _multiplayerRoots[index];

            var inputModule = esGo.AddComponent<InputSystemUIInputModule>();

            // 尝试由 EUInputController Generated 提供 Asset（InputControllerAsset 模式）
            InputActionAsset resolvedAsset = null;
            string resolvedMapName = null;
            OnResolveMultiplayerPlayerAsset(index, ref resolvedAsset, ref resolvedMapName);

            if (resolvedAsset != null)
            {
                // InputControllerAsset 模式：直接使用 PlayerInputController 自带的 Asset，
                // 设备隔离由 PlayerInputController.BindGamepad() 自动维护，EUUI 不克隆也不做设备管理。
                var activeMap = resolvedAsset.FindActionMap(resolvedMapName);
                inputModule.actionsAsset = resolvedAsset;
                if (activeMap != null)
                    BindInputModuleActions(inputModule, activeMap);

                _multiplayerPlayerAssets[index] = resolvedAsset;
            }
            else if (sourceAsset != null)
            {
                // MultiplayerUIEvent 模式：克隆 Asset，各玩家持有独立实例，使 Map 启用状态互不影响
                var playerAsset = UnityEngine.Object.Instantiate(sourceAsset);
                string mapName = MultiplayerMapNames[index];

                // 禁用所有 Map，仅启用本玩家的 Map → 确保只有本玩家的按键路径会触发 Action
                foreach (var map in playerAsset.actionMaps)
                    map.Disable();
                var activeMap = playerAsset.FindActionMap(mapName);
                activeMap?.Enable();

                // 将克隆 Asset 的 Action 槽手动绑定到 InputSystemUIInputModule
                inputModule.actionsAsset = playerAsset;
                if (activeMap != null)
                    BindInputModuleActions(inputModule, activeMap);
```

```

                _multiplayerPlayerAssets[index] = playerAsset;
            }

            _multiplayerEventSystems[index] = mpEventSystem;

            // 初始禁用：仅在 EnterMultiplayerMode 时才激活，避免与全局 EventSystem 竞争
            esGo.SetActive(false);
        }



```

OnResolveMultiplayerPlayerAsset获取EUInputController中获取的方式

C#

```
        /// <summary>
        /// InputControllerAsset 模式下的 Asset 解析钩子。
        /// 将对应槽位 PlayerInputController 自带的 InputController Asset 及 UI Map 名称填入输出参数；
        /// MultiplayerUIEvent 模式下不做任何操作（保持 null，由 CreatePlayerEventSystem 走默认分支）。
        /// </summary>
        static partial void OnResolveMultiplayerPlayerAsset(int slotIndex, ref InputActionAsset asset, ref string mapName)
        {
            if (Config.multiplayerInputMode != MultiplayerInputMode.InputControllerAsset) return;

            var controllers = EUInputController.GetPlayerInputControllerList();
            if (slotIndex >= controllers.Length) return;

            asset   = controllers[slotIndex].Controller.asset;
            mapName = "UI";
        }
```

因为不是默认的UI这个map或者是直接使用defaultInputAssets所以需要自己进行处理槽位的绑定

C#

```
        /// <summary>
```

```
        /// 将 actionMap 中的标准 UI Action 绑定到 InputSystemUIInputModule 的对应槽位。
        /// </summary>
        private static void BindInputModuleActions(InputSystemUIInputModule inputModule, InputActionMap map)
        {
            TryBind(map, "Navigate",                 a => inputModule.move = a);
            TryBind(map, "Submit",                   a => inputModule.submit = a);
            TryBind(map, "Cancel",                   a => inputModule.cancel = a);
            TryBind(map, "Point",                    a => inputModule.point = a);
            TryBind(map, "Click",                    a => inputModule.leftClick = a);
            TryBind(map, "MiddleClick",              a => inputModule.middleClick = a);
            TryBind(map, "RightClick",               a => inputModule.rightClick = a);
            TryBind(map, "ScrollWheel",              a => inputModule.scrollWheel = a);
            TryBind(map, "TrackedDevicePosition",    a => inputModule.trackedDevicePosition = a);
            TryBind(map, "TrackedDeviceOrientation", a => inputModule.trackedDeviceOrientation = a);
        }
        private static void TryBind(InputActionMap map, string actionName, System.Action<InputActionReference> setter)
        {
            var action = map.FindAction(actionName);
            if (action != null)
                setter(InputActionReference.Create(action));
        }

```

1. 在 map 里找到这个行为（InputAction）
2. 用 InputActionReference.Create(action) 创建该行为的引用
3. 把这个引用赋给 inputModule 的对应属性（move / submit / point / leftClick 等）

到这一步通过map，只解决了p1 p2的键盘输入问题，手柄的处理需要解决playerAsset做devices设置

分配手柄

分为两种情况：

使用EUInputController的InputAction Assets

直接使用EUInputController的InputAction Assets，设备分配由EUInputController处理

使用Resources.Load

_multiplayerPlayerAssets，创建时以及缓存了assets只需要，通过GetValidPlayerAsset，传入玩家index（0~3）获取到对应的assets

```C#
        private static InputActionAsset GetValidPlayerAsset(int index)
        {
            if (_multiplayerPlayerAssets == null || index < 0 || index >= _multiplayerPlayerAssets.Length)
                return null;
            return _multiplayerPlayerAssets[index];
        }
```

调用AssignGamepadPlayer，传入index 和设备就可以让该assets只接受对应设备的输入，这部分由管理设备的部分进行调用

```C#
        /// <summary>
        /// 为指定玩家槽位分配键盘
        /// </summary>
        public static void AssignKeyboardPlayer(int index)
        {
            var asset = GetValidPlayerAsset(index);
            if (asset == null) return;

            var list = new List<InputDevice>(2);
            if (Keyboard.current != null) list.Add(Keyboard.current);
            if (Mouse.current != null) list.Add(Mouse.current);
            asset.devices = list.Count > 0 ? list.ToArray() : null;
            Debug.Log($"[EUUIKit.Multiplayer] P{index + 1} → 键盘区域 {MultiplayerMapNames[index]}（仅键鼠，不接收手柄）");
        }
        /// <summary>
        /// 为指定玩家槽位分配手柄
        /// </summary>
        public static void AssignGamepadPlayer(int index, Gamepad gamepad)
        {
```

```
            var asset = GetValidPlayerAsset(index);
            if (asset == null || gamepad == null) return;

            // 限定 Asset 只接受来自此手柄的输入，实现手柄的独占分配
            asset.devices = new InputDevice[] { gamepad };
            Debug.Log($"[EUUIKit.Multiplayer] P{index + 1} → 手柄 {gamepad.displayName}（独占锁定）");
        }

```

从EUInputController获取设备监听的变化情况

C#

```
        static partial void OnMultiplayerInitialized(int playerCount)
        {
            if (Config.multiplayerInputMode == MultiplayerInputMode.MultiplayerUIEvent)
            {
                EUInputController.AddPlayerInputDeviceAddedListener(OnMultiplayerGamepadAdded);
                EUInputController.AddPlayerInputDeviceRemovedListener(OnMultiplayerGamepadRemoved);
            }
        }
```

外部调用一次分配默认设备

C#

```
        public static void AssignDefaultDevices()
        {
            if (!_multiplayerInitialized)
            {
                Debug.LogWarning("[EUUIKit.Multiplayer] 请先调用 InitMultiplayerEventSystem()");
                return;
            }

            if (Config.multiplayerInputMode == MultiplayerInputMode.InputControllerAsset)
            {
```

```
                Debug.Log("[EUUIKit.Multiplayer] InputControllerAsset 模式：设备由 PlayerInputController 自管，AssignDefaultDevices 为空操作。");
                return;
            }

            AssignDevicesByPriority(_multiplayerPlayerCount);
        }
```

进行分配处理

C#

```
        private static void AssignDevicesByPriority(int playerCount)
        {
            if (playerCount <= 0) return;

            // 从 EUInputController 获取玩家控制器列表（按加入顺序，代表逻辑上的"几号玩家"）
            var controllers = EUInputController.GetPlayerInputControllerList();

            int gamepadSlots  = 0;
            int keyboardSlots = 0;

            for (int i = 0; i < playerCount; i++)
            {
                Gamepad gamepad = i < controllers.Length ? controllers[i].Gamepad : null;

                if (gamepad != null)
                {
                    AssignGamepadPlayer(i, gamepad);
                    gamepadSlots++;
                }
                else
                {
                    AssignKeyboardPlayer(i);
                    keyboardSlots++;
                }
            }

            Debug.Log($"[EUUIKit.Multiplayer] 设备分配完成：键盘槽={keyboardSlots}（Map隔离），手柄槽={gamepadSlots}，总玩家={playerCount}");
        }

```